

Cs 420 draft final project:

1. ChromaCode — a programming language where colors represent logic

Idea:

ChromaCode is a programming language where **colors are part of the syntax**, not just decoration. Instead of only reading words and symbols, the program can understand meaning from color.

For example:

- **Blue** = variables / data
- **Green** = functions / output
- **Yellow** = conditions
- **Purple** = loops / repetition
- **Red** = stop / error

So in ChromaCode, code becomes more visual. A programmer could still type words, but the **color attached to each command helps show what the command does**.

What problem it solves

ChromaCode could make coding:

- more visual for beginners
- easier to scan quickly
- useful for educational programming
- helpful on touchscreens or block-based coding systems
- more creative by combining programming with design and images

Syntax and Semantics

A color tag must be before its keyword on the same line. Indentation defines scope similar to python. A color mismatch such as using a YELLOW tag on a non boolean expression would throw a ColorTypeError

ChromaCode uses lexical scoping, Data types are inferred and include floats, strings, integers, and booleans. Execution is sequential by default. GREEN tagged expressions support basic functions such as lamdas and return values. Color mismatches are caught at compile time by ColorTypeError.

Features

- Supports imperative and light functional programming

- Dual channel syntax with color tag and text keyword having independent meaning
- Compile time has color role enforcement with ColorTypeError's
- Visual image viewing through a strip of colored pixels in a PNG compiles to executable code
- 1-1 mapping with Scratch style block types allowing new programmers to visually interpret code

Extra feature: coding from pictures/colors

One unique idea for ChromaCode is that it could let users **insert a picture or colored pattern**, and the language would translate those colors into code.

For example, if the system reads a sequence of colors from an image:

- **Purple** = repeat
- **Yellow** = check condition
- **Green** = print output
- **Red** = stop

then a colored image could generate a program such as **FizzBuzz**.

A strip of colored squares might mean:

- Purple square = loop through numbers 1 to 100
- Yellow square = if number divisible by 3
- Green square = print "Fizz"
- Yellow square = if number divisible by 5
- Green square = print "Buzz"
- Yellow square = if divisible by both
- Green square = print "FizzBuzz"
- Red square = end

That means a user could almost "paint" a program instead of typing it all out.

This would make programming feel more interactive and could help students understand patterns in logic by seeing them visually.

Example Programs

Text-style ChromaCode could look like this:

FizzBuzz

```
PURPLE for i in 1..100
  YELLOW if i % 15 == 0
```

```
    GREEN print("FizzBuzz")
YELLOW elif i % 3 == 0
    GREEN print("Fizz")
YELLOW elif i % 5 == 0
    GREEN print("Buzz")
YELLOW else
    GREEN print(i)
RED end
```

Hello World

```
BLUE name = "World"
YELLOW if name != ""
    GREEN print("Hello, " + name)
RED end
```

Define function and calling function

```
GREEN func add(a, b)
    BLUE result = a + b
    GREEN return result
RED end
```

```
BLUE total = add(3, 7)
GREEN print(total) // outputs: 10
```

A more advanced version could let the **actual color itself** be the command, so the computer reads the visual color and not just the word.

Ecosystem

- ChromaCode IDE plugin- VSCode extension that would render color tags live in the editor, making the dual channel syntax visible as you type. Also includes an image importer that converts a sequence of colored squares into ChromaCode source code.
- Compiler- Compiler that uses visual flags to output a color annotated HTML render of source code. Helps students and instructors walk through code.
- Standard Library- A group of built in modules that cover I/O, math, string manipulation, and collections. Every public function signature is tagged with its color role so that developers can easily and quickly identify what type of operation they are working with.

- Website- browser based IDE that requires no installation. Users can type ChromaCode directly or paint colored squares on a grid to create and run interactive programs, making it an entry point for beginners and for educators to use.

Challenges

There are a few potential challenges including

- Color blindness- to help with this we would use shapes or font styles to help differentiate between colors
- Standardization across hardware- screens can render hues and colors differently so the language must have strict RGB tolerance
- Complexity- the image processing and color classification is an additional layer that can be hard to perfect
- Color tags would be beneficial for beginners but may make it restrictive for advanced programmers (similar to how scratch is difficult when making complex programs)